

Java String Immutability - Cheat Sheet

A `String` in Java can't change once it's created. Every operation that looks like it modifies a string actually returns a brand-new one. This sounds wasteful until you see what the JVM does with it: pooling, hash caching, free thread safety, and a chunk of security guarantees.

What “Immutable” Actually Means

A `String` object holds a `char[]` (or `byte[]` since Java 9, with compact strings). That backing array is `final` and `private`, and no method on `String` exposes it. Once a string is built, its contents are locked.

```
String s = "hello";
s.toUpperCase();    // returns "HELLO", doesn't change s
System.out.println(s); // still "hello"

s = s.toUpperCase(); // now s points to a new String
System.out.println(s); // "HELLO"
```

The variable `s` can point to a different `String`. The original `"hello"` object never changed.

Why Java Made String Immutable

Four practical reasons.

1. String pool / caching. Identical string literals share one object in memory. If strings were mutable, one piece of code could change a literal and silently break unrelated code that points to the same instance.

```
String a = "hello";
String b = "hello";
System.out.println(a == b); // true, same object in the pool
```

2. Thread safety for free. Nothing to mutate means nothing to race on. You can share a `String` across threads with zero locking.

3. Hashcode caching. `String.hashCode()` is computed once and cached on the instance. That's why `HashMap` with `String` keys is fast.

4. Security. Strings carry passwords, URLs, file paths, class names. If a `String` could be modified after a security check, a thread could swap the value between check and use. Immutability closes that window.

The String Pool

Java keeps a special area in the heap, the string constant pool, for string literals.

```
String a = "hello";           // goes in the pool
String b = "hello";           // reuses the same pool entry
String c = new String("hello"); // forces a fresh object on the heap

System.out.println(a == b);    // true
System.out.println(a == c);    // false
System.out.println(a.equals(c)); // true
```

Rule of thumb: always use `.equals()` to compare content. `==` only checks whether two references point to the same object.

`==` vs `.equals()` Quick Reference

What you write	What it checks
<code>a == b</code>	Same object reference
<code>a.equals(b)</code>	Same content
<code>a.equalsIgnoreCase(b)</code>	Same content, ignoring case
<code>a.compareTo(b)</code>	Lexicographic order

`==` gives surprising results once `new String(...)` enters the picture. Stick with `.equals()` for content.

`intern()`

Forces a string onto the pool. Useful when you're producing lots of identical strings at runtime and want them to share memory.

```
String a = new String("hello");
String b = a.intern(); // returns the pool's "hello"
String c = "hello";

System.out.println(a == c); // false
System.out.println(b == c); // true
```

Use sparingly. The pool sits in the heap since Java 7, but interning still has a cost. Don't intern unbounded user input.

Operations That Look Like Mutations

Every “modifying” method on `String` returns a new object. The original is untouched.

```
String s = "hello world";

s.toUpperCase();    // returns "HELLO WORLD"
s.replace('o', '0'); // returns "hell0 w0rld"
s.substring(0, 5);  // returns "hello"
s.concat(" again"); // returns "hello world again"
s.trim();           // returns "hello world" (already trimmed)

System.out.println(s); // still "hello world"
```

If you want the result, capture it:

```
String upper = s.toUpperCase();
```

When You Actually Need a Mutable String

Two options, both share the same API.

Class	Thread-safe?	Use when
<code>StringBuilder</code>	No	Single-threaded, the common case
<code>StringBuffer</code>	Yes (synchronized)	Multi-threaded, rare in practice

Example:

```
StringBuilder sb = new StringBuilder();
sb.append("hello");
sb.append(", ");
sb.append("world");
sb.insert(0, ">>> ");
sb.reverse();
System.out.println(sb.toString());
```

Pick `StringBuilder` by default. Reach for `StringBuffer` only when multiple threads are appending to the same buffer, which is uncommon.

String Concatenation Costs

A loop with `+` on strings looks innocent but creates a new `String` on every iteration.

Bad:

```
String result = "";  
for (int i = 0; i < 1000; i++) {  
    result += i; // builds a new String each pass  
}
```

Good:

```
StringBuilder sb = new StringBuilder();  
for (int i = 0; i < 1000; i++) {  
    sb.append(i);  
}  
String result = sb.toString();
```

The compiler converts simple inline concatenations like `"a" + "b" + "c"` into a single `StringBuilder` call for you. The loop case is where it can't help, and that's where the cost piles up.

Common Gotchas

- `"hello".replace('l', 'L')` doesn't change anything in place. Catch the return value.
 - `s.trim()` returns the same object when there's nothing to trim.
 - `new String("hello")` is almost always a mistake. It forces an extra object and skips the pool.
 - `String.format(...)` and `String.join(...)` look like they modify something, but they're static and return new strings.
 - For sensitive data like passwords, prefer a `char[]` you can zero out after use. A `String` lingers in the pool until GC reclaims it.
 - `intern()` on uncontrolled user input can balloon the heap. Bound the input set first.
 - Two strings with the same content but built differently can fail `==`. Always reach for `.equals()`.
-

Quick Reference

Concept	Key fact
Immutability	Backing array is <code>final</code> and <code>private</code>
String pool	Identical literals share one instance
<code>==</code>	Reference equality, not content
<code>.equals()</code>	Content equality, the safe default
<code>intern()</code>	Forces a string into the pool
<code>StringBuilder</code>	Mutable, single-threaded
<code>StringBuffer</code>	Mutable, synchronized
<code>+</code> in a loop	Use <code>StringBuilder</code> instead